

Understanding Large Language Models

LLM 이해

LLM 활용

목차 LLM 활용

1. API

2. LLM API

3. 하이퍼 파라미터

1. API

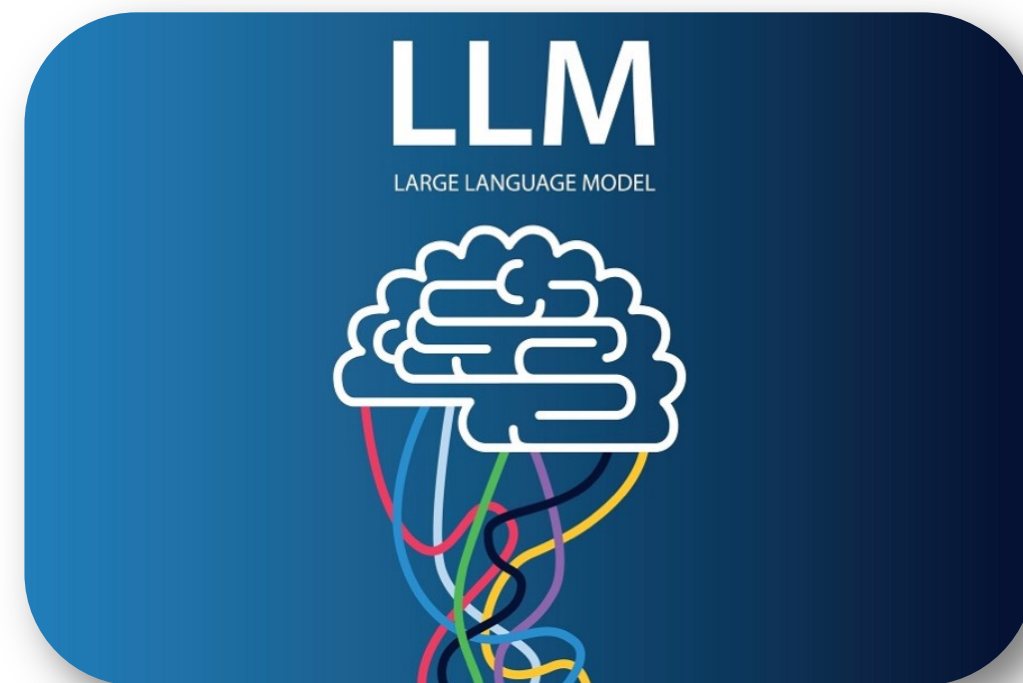
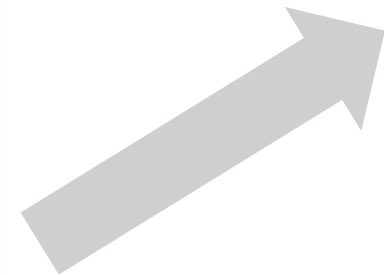
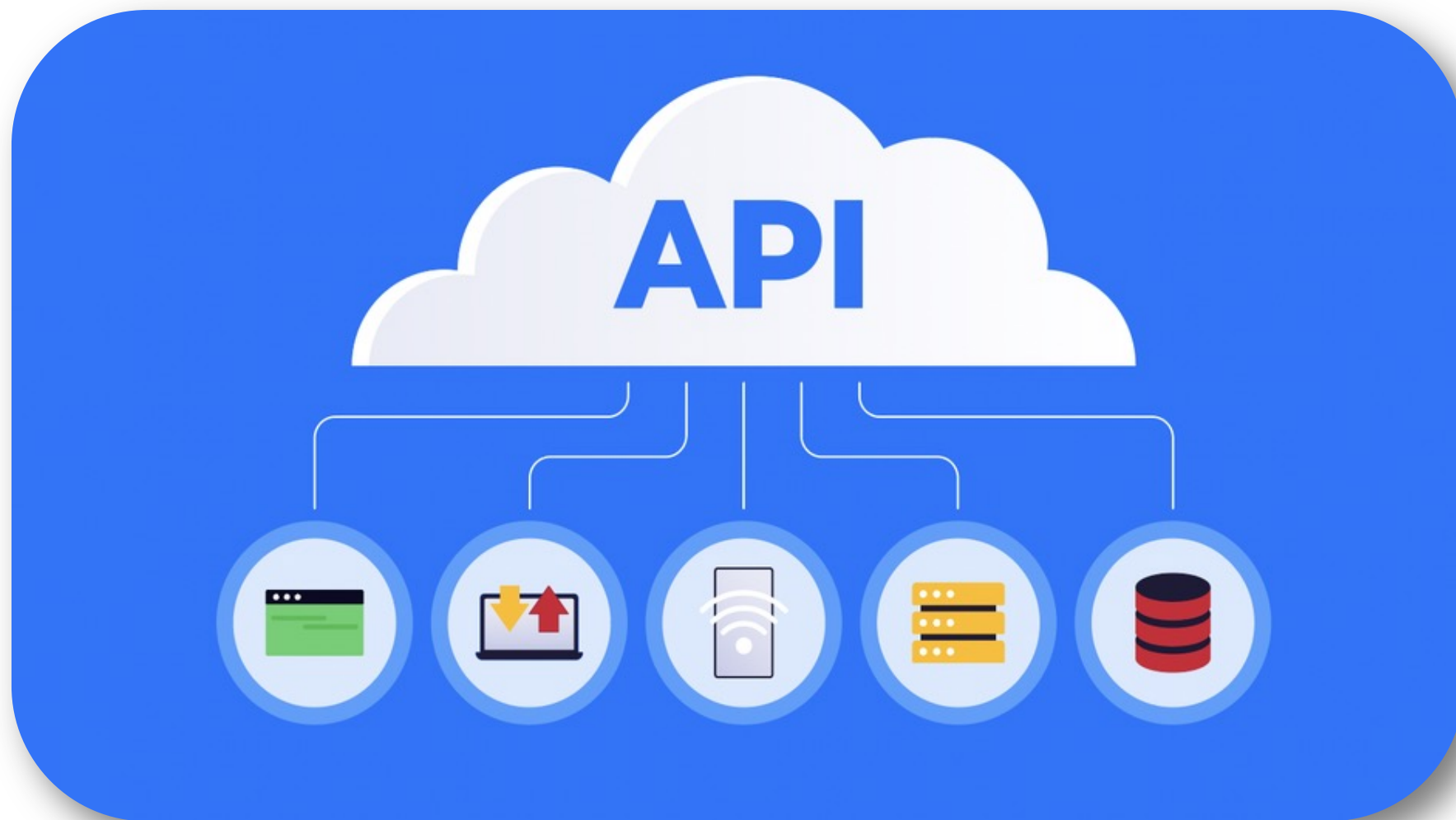


API란 무엇인가

API (Application Programming Interface)

소프트웨어 간 상호작용을 가능하게 하는
인터페이스

- API(Application Programming Interface)는 서로 다른 시스템이나 서비스가 정보나 기능을 주고받을 수 있도록 만든 규칙
- API의 종류는 LLM API와 HTTP API가 있음





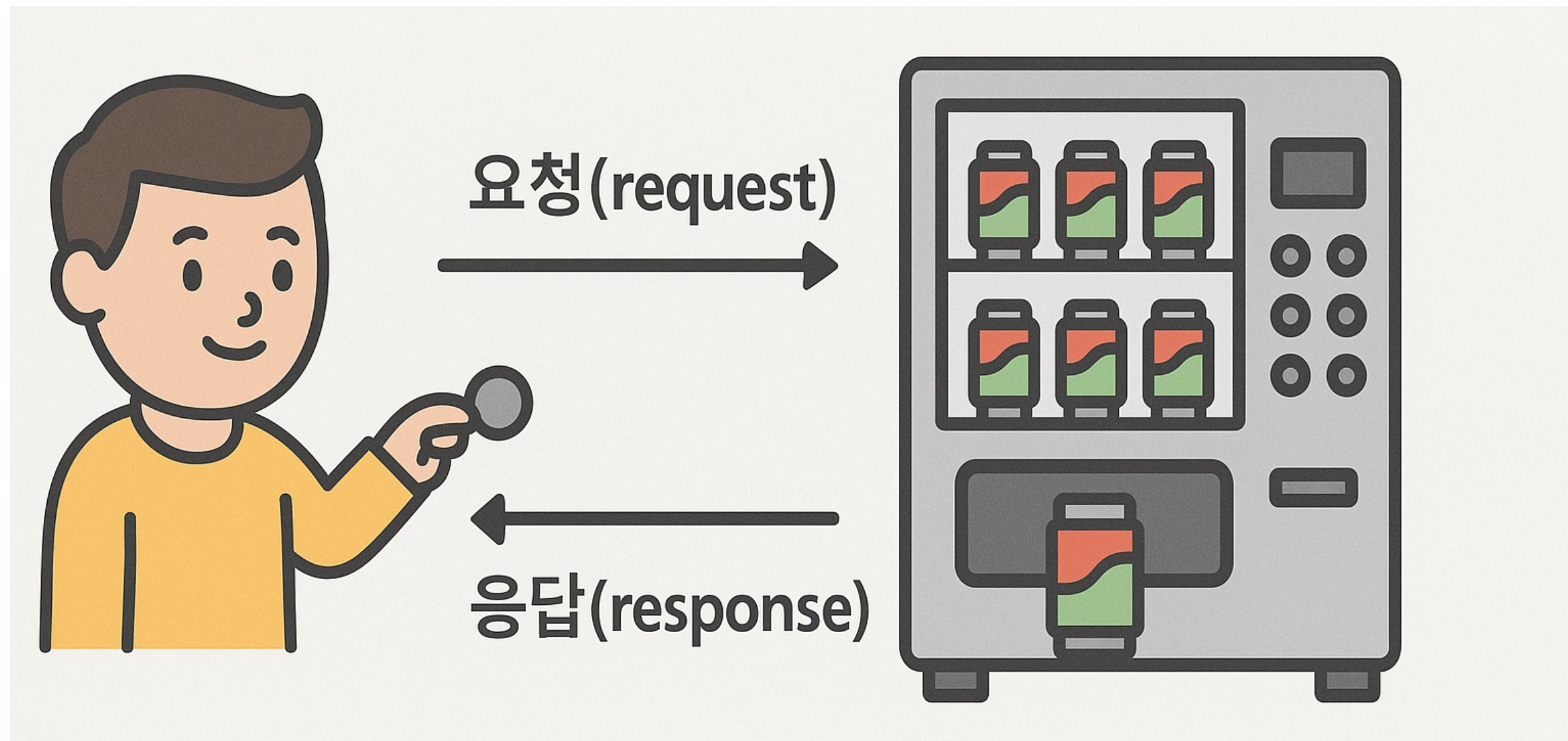
API의 개념

Application Programming Interface

응용 프로그램이 서로 통신하고 데이터를 교환할 수 있도록 하는 규칙과 약속

API의 개념

클라이언트가 **요청(request)**를 보내면 서버에서 **처리**를 하고 **응답(response)** 전달



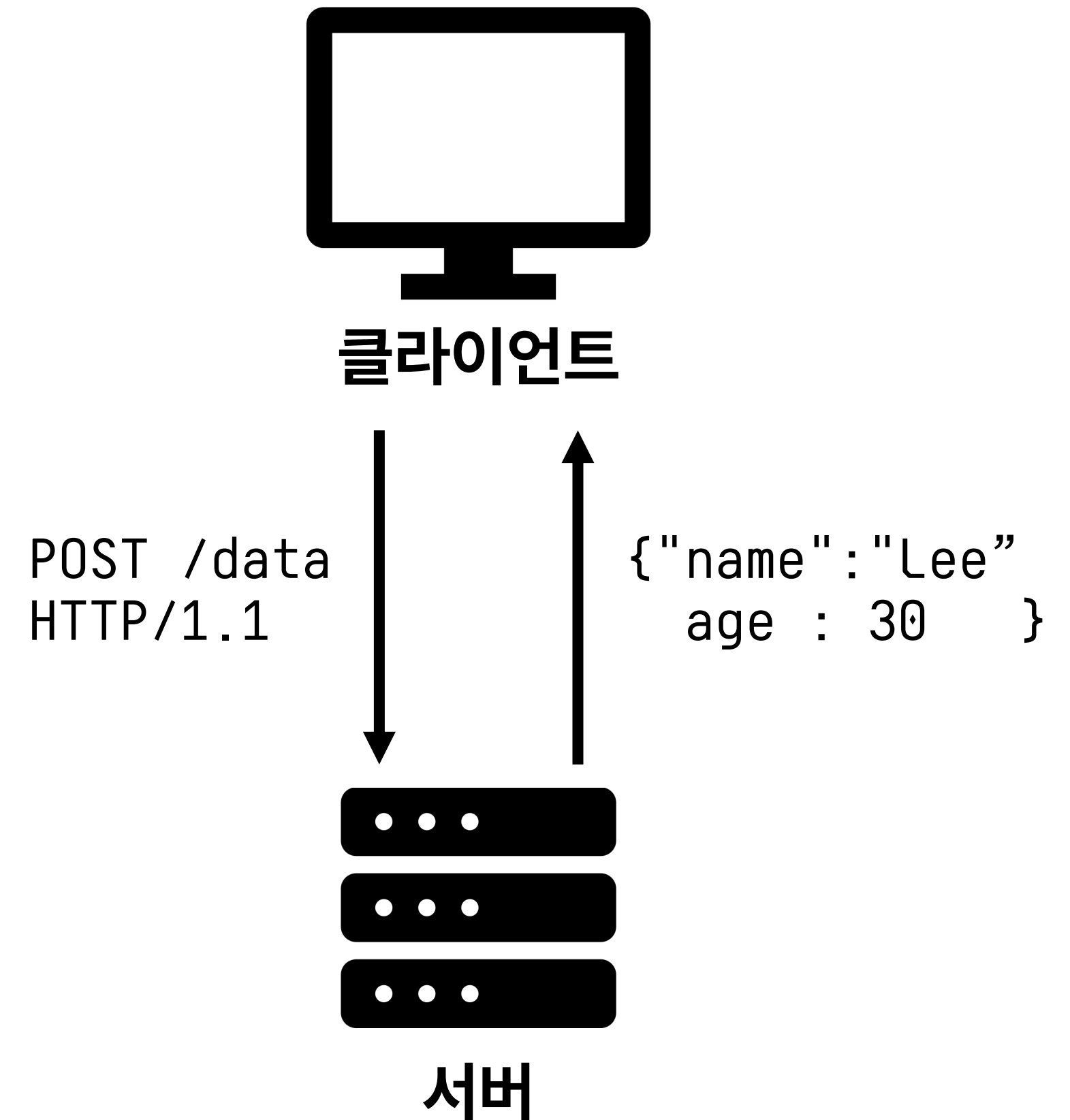
API의 중요성

- API는 소프트웨어 간 연결을 위한 표준화된 창구
- 오늘날 모든 디지털 서비스(앱, 웹, AI, IoT)는 **API를 통해** 상호작용
- 구성 요소 간 의존도를 낮추면서도 유기적인 협업 가능

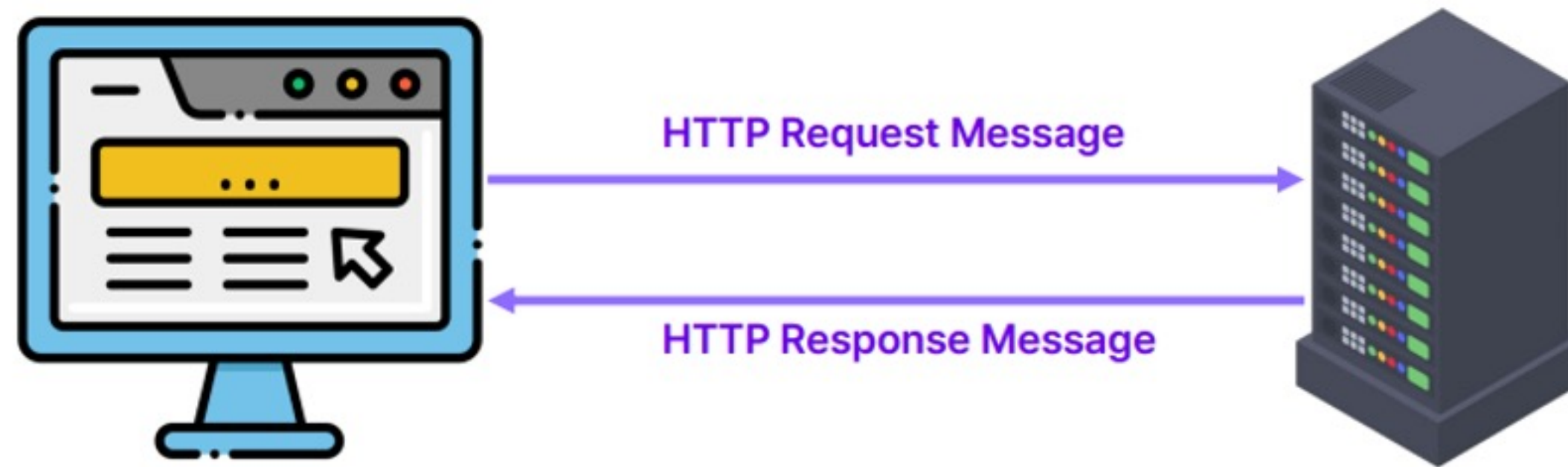
서비스	API 역할
모바일 앱	날씨, 뉴스, 로그인 기능 모두 외부 API 호출
서드파티 로그인	OAuth API로 사용자 인증 처리
챗봇/AI 비서	LLM API로 텍스트 생성 응답 받기
결제 시스템	카드사 API로 연동
스마트홈	IoT API로 기기 제어 (전등, 온도 등)

API 요청과 응답 기본 구조

- 요청(request)과 응답(response)을 주고받는 구조
- 클라이언트가 서버에 데이터를 요청하면,
서버는 데이터를 JSON/XML 등으로 응답
- HTTP 프로토콜을 사용하는 경우가 많음
(특히 웹 기반 API)



HTTP 프로토콜



- 클라이언트-서버 간 데이터가 교환되는 방식에 대한 규약
- 거의 모든 형태의 데이터 전송 가능
- 무상태성과 비연결성



HTTP API

특정 기능/데이터를 요청-응답 방식으로 제공하는 API

- HTTP 프로토콜을 기반으로 한 정보 교환 방식
- 주소(URL)와 정해진 규칙을 이용해 데이터를 요청하면, 서버가 그에 맞는 응답을 반환
- 특히 RESTful API는 단순하고 일관된 구조를 사용해, 다양한 서비스와 쉽게 연결되고 활용 가능

• 점원의 역할



• API의 역할



API에서 가장 많이 쓰이는 데이터 형식 – JSON

- 데이터를 사람이 읽을 수 있는 형태로 구조화하는 형식
- 대부분의 API는 요청과 응답 데이터를 **JSON** 형태로 주고받음
- **키(Key)-값(Value)** 쌍으로 이루어짐
- 각 키는 문자열로 표현되며 키와 값은 콜론(:)으로 구분됨
- 지원 데이터 타입
 - 문자열 (" ")
 - 숫자
 - 객체 ({ })
 - 배열 ([])
 - boolean / null

```
{
  "name": "Lee",
  "age": 30,
  "roles": ["user", "editor"],
  "preferences": {
    "theme": "dark",
    "language": "ko"
  },
  "login_history": [
    {"date": "2025-05-01", "ip": "192.168.1.10"},
    {"date": "2025-05-07", "ip": "192.168.1.11"}
  ],
  "score": null
}
```

API 요청과 응답 기본 구조 (일반 API)

요청

- **HTTP Method** : 요청 방식 (GET, POST 등)
 - GET : 데이터 조회
 - POST : 데이터 생성
 - DELETE : 데이터 삭제
- **Endpoint (URL)** : 데이터를 요청할 서버의 주소
 - 종종 버전 정보(/v1)나 리소스 ID(/users/123)가 포함

POST https://api.example.com/userdata

Content-Type: application/json

Authorization: Bearer abc123

```
{  
  "user_id": "001",  
  "detail": true  
}
```

API 요청과 응답 기본 구조 (일반 API)

요청

- **Headers** : 인증 정보, 데이터 형식 등
 - **Content-Type** : 본문(body) 데이터의 형식
 - **Authorization** : 인증용 토큰
- **Body** : 전달할 실제 데이터 (JSON, XML 등) ←
 - 대부분 JSON 형식으로 전송되며,
사용자가 요청한 리소스나 처리 결과를 포함

POST https://api.example.com/userdata

Content-Type: application/json

Authorization: Bearer abc123

```
{  
  "user_id": "001",  
  "detail": true  
}
```

API 요청과 응답 기본 구조 (일반 API)

응답

- **Status Code** : 처리 결과 (200, 400, 500 등)
 - HTTP/1.1 : 사용 중인 HTTP 프로토콜 버전
 - 200 : 요청이 성공적으로 처리됨
- **Headers** : 응답 형식, 길이 등
 - Content-Type : JSON
 - Content-Length : 응답 JSON이 52바이트 길이
- **Body** : 실제 응답 데이터

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 52

{
  "name" : "Lee",
  "age" : 30,
  "membership" : "premium"
}
```

대표적인 Status Code

	의미	예시
200	OK	요청 정상 처리됨
201	Created	리소스 성공적으로 생성됨 (POST)
204	No Content	응답 본문 없음 (삭제 성공 등)
400	Bad Request	요청 형식 오류, 필수 파라미터 누락
401	Unauthorized	인증 실패 (토큰 없음/만료 등)
403	Forbidden	권한 없음
404	Not Found	해당 리소스 없음
500	Internal Server Error	서버 내부 오류

API 요청과 응답 기본 구조 (RESTful API)

- **REST** = **R**epresentational **S**tate **T**ransfer
- 리소스를 URL로 표현하고,
이를 HTTP 메서드(GET, POST 등)로 조작
- HTTP를 기반으로
리소스를 명확하게 식별하고, 상태를 표현하는 방식
- 클라이언트-서버 구조에서 **무상태성(Stateless)**을 강조
- 대부분의 LLM API는 REST 방식 기반

```
POST
https://api.openai.com/v1/chat/completions
Authorization: Bearer <API_KEY>
Content-Type: application/json

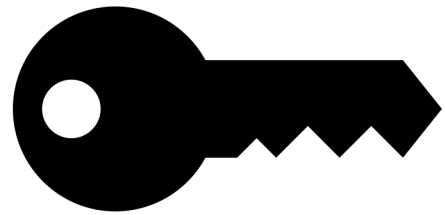
{
  "model": "gpt-4o",
  "messages":
    [{"role": "user",
      "content": "Tell me a joke."}],
  "temperature": 0.7
}
```

일반 API VS RESTful API

	일반 API	RESTful API
구조	자유롭거나 불명확함	일관된 구조
HTTP 메서드	보통 POST만 사용	GET/POST/PUT/DELETE 등 기능에 따라 분리하여 사용
상태 유지 (Stateless)	가능	각각의 요청은 독립적이며 서버는 이전 상태를 기억하지 않음 (Stateless 원칙)
형식 통일성	없음	JSON + 표준 State 코드 + 명확한 응답 구조 추천

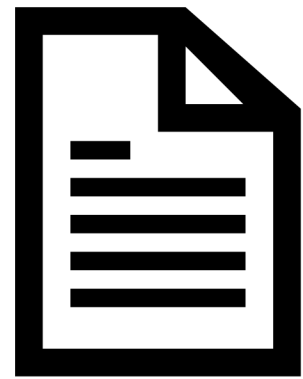


API 사용 시 유의사항



API KEY

외부에 노출되면
비용 부과·보안 위험이
발생할 수 있음



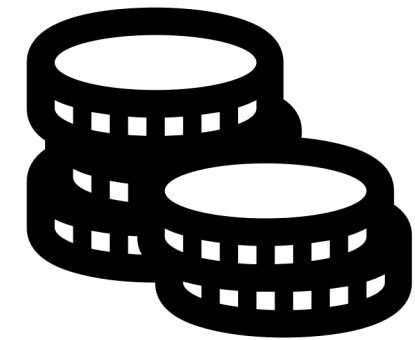
.env

API Key나
민감한 설정값은
.env 파일에 저장



Rate Limit

대부분의 API 서비스는
과도한 요청 방지를
위해 호출 횟수 제한이
있음



토큰 기반 과금

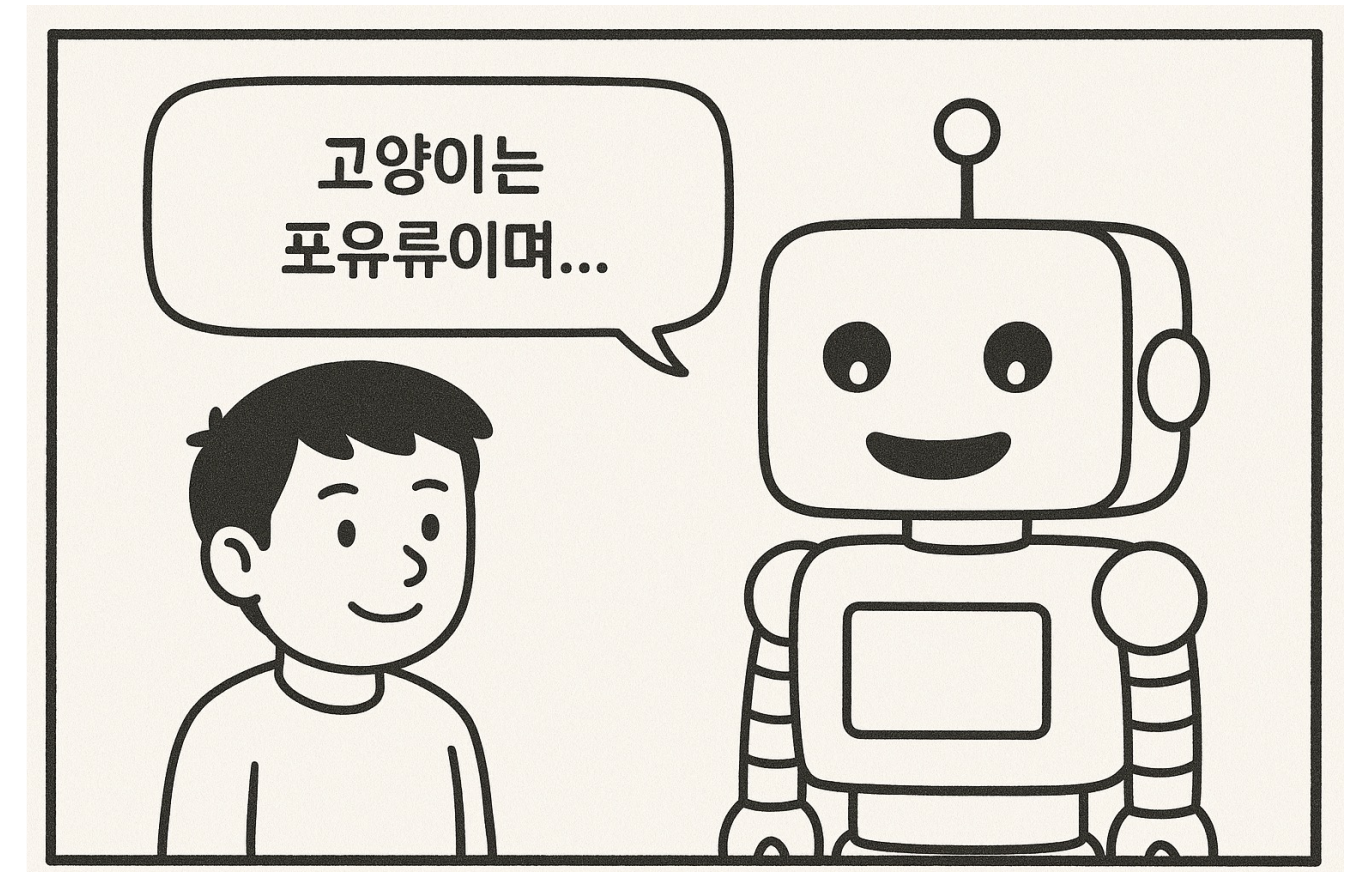
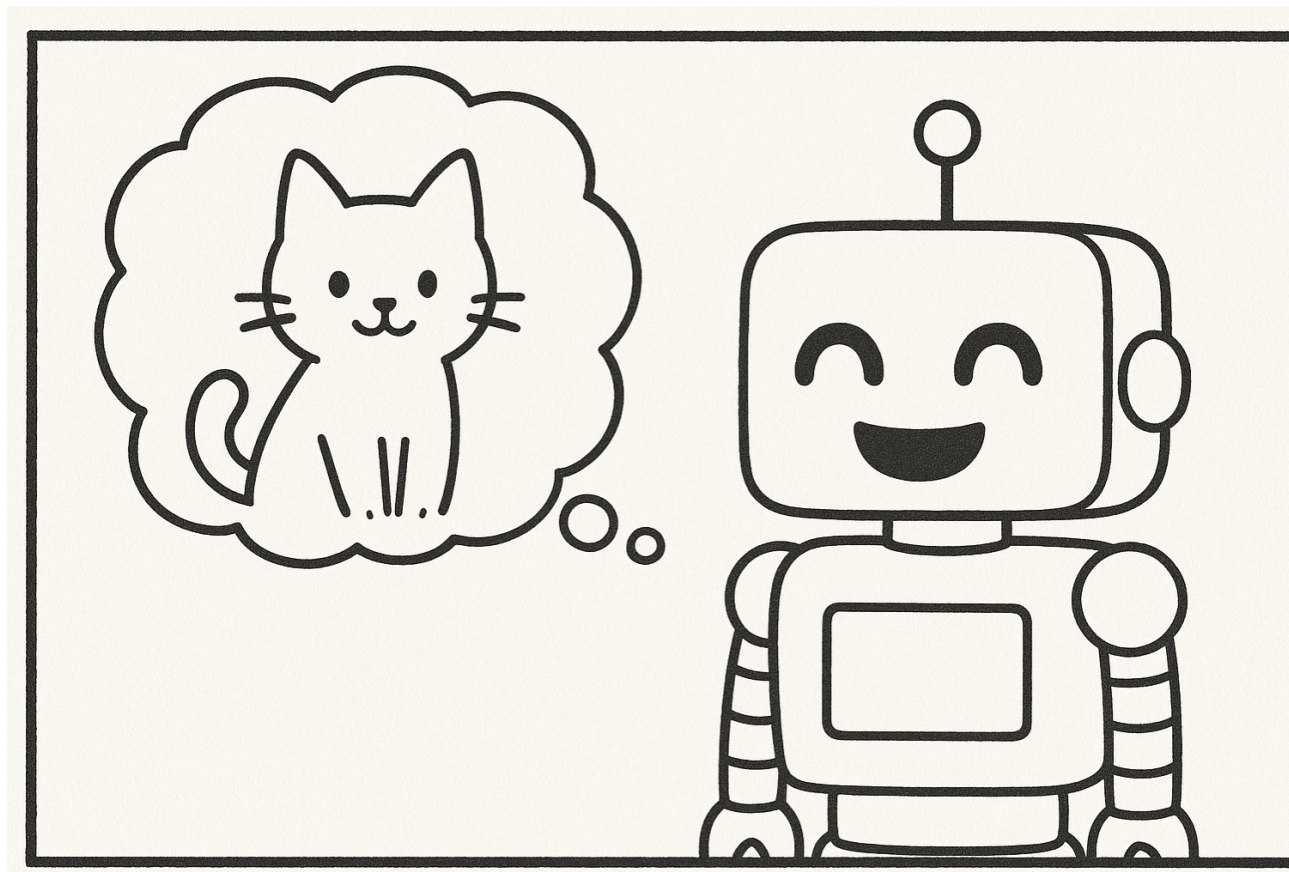
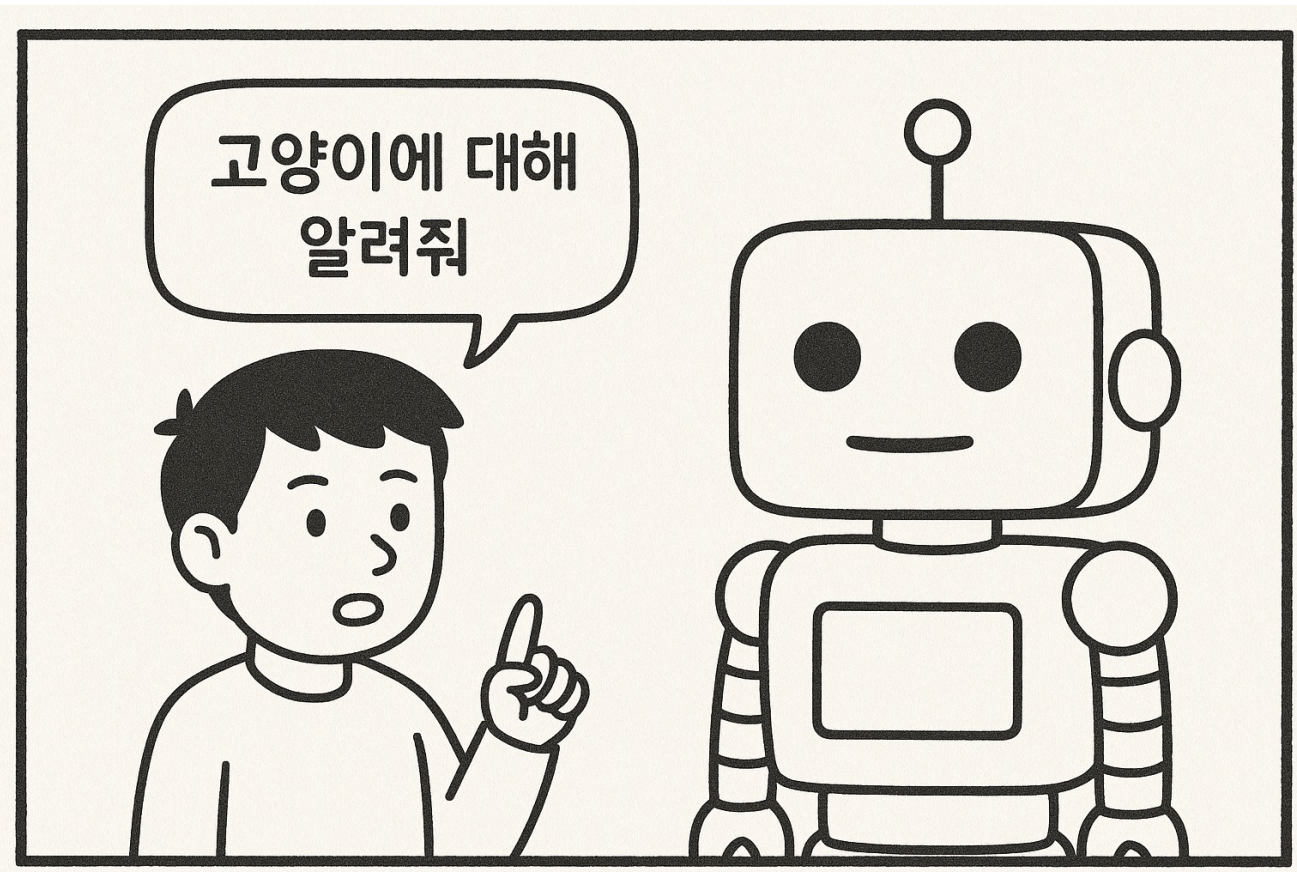
요청 수가 적어도
토큰 수가 많으면
큰 요금이 발생할 수
있음

2. LLM API

LLM API 개념

텍스트 입력에 대해 텍스트를 생성하는 API

- 사용자는 **프롬프트(prompt)**를 입력
- 모델은 문맥에 맞는 텍스트를 **생성(generative)**
- 전통적인 API와 달리 고정된 정답이 아닌 확률 기반의 **다양한 응답**이 가능



LLM API의 작동 방식

요청

1. **model** : 모델 지정 (gpt-4, llama 등)
2. **messages** : 프롬프트 입력
3. **파라미터 설정**
 - **temperature** : 응답의 무작위성/창의성 제어
 - **max_tokens** : 출력 길이 제한

응답

- 생성된 텍스트로 반환 (보통 JSON 형태)

```
{  
  "model": "gpt-4o-mini",  
  "messages":  
    [{"role": "user",  
      "content": "한국의 수도는 어디야?"}],  
  "temperature": 0.5  
}
```

전통 API VS LLM API

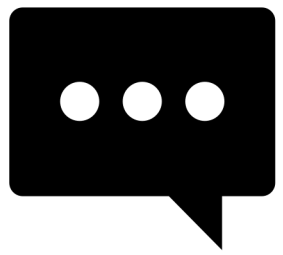
	일반 API	LLM API
응답 형식	정형화된 응답 (JSON, XML 등)	유동적인 텍스트 생성
예측 가능성	동일한 요청에 항상 동일한 응답 제공	동일한 요청에 응답이 매번 달라질 수 있음
활용 목적	CRUD, 정해진 기능 호출	정보 생성, 요약, 번역 등 자연어 처리
설계 복잡도	명확한 구조 필요	프롬프트 구성과 파라미터 조정 필요

주요 LLM API 플랫폼 비교

	주요 모델	특징	멀티 모달 지원	최대 컨텍스트
OpenAI	- GPT-4o - GPT-5.2 - o3	- 멀티모달 입력/출력 - 고성능 추론 - 광범위한 API 생태계	가능	최대 ~400K tokens
Google (Gemini)	- Gemini 3 Pro - Gemini 3 Flash	- 초대형 컨텍스트 처리 - Google 생태계 연계 - 빠른 반응 속도	가능	약 1M tokens
Anthropic	- Claude Opus 4.5 - Claude Sonnet 4.5 - Claude Haiku 4.5	- 장기/복잡 워크플로우에 강함 - 높은 신뢰도 - 문서 기반 분석에 탁월	제한적 (텍스트, 이미지)	200k tokens 또는 1M tokens
Hugging Face	- DeepSeek-R1 - Qwen 3 - Mistral	- 개발자 친화적 플랫폼 - 다양한 오픈소스 LLM 제공 - 자유로운 커스터마이징 가능	모델별 상이	모델별 상이
Meta	- LLaMA 4 Maveric - LlaMA 4 Scout	- 오픈소스 기반 API 출시 - 고성능 언어 모델을 자체적으로 활용 가능	제한적 (텍스트, 이미지)	약 128k tokens

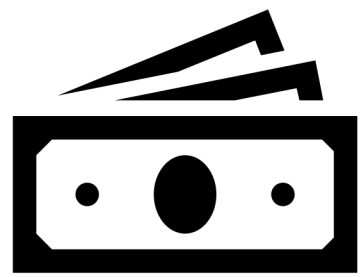


LLM API 사용 시 유의사항



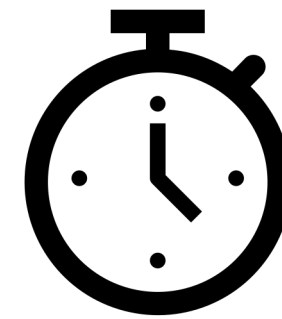
비결정성

같은 질문에도
답이 매번 다를 수 있음



비용 발생

토큰 사용량에 따라
과금됨



속도와 응답 시간

모델의 크기나
부하 상태에 따라
응답이 느릴 수 있음



보안

입력한 프롬프트는
외부 서버로 전송
**민감한 정보는
입력 금지**

비용 발생 구조

테스트 시에도 실사용 요금이 부과될 수 있음
→ free quota / trial limit 체크 필요

- LLM API는 사용량 기반 과금이라
요청 수가 적어도 토큰 수가 많으면 큰 요금이 발생할 수 있음
- 요청마다 총 토큰 수를 로그에 남겨서 모니터링해야 함

Flagship models

Our frontier models are designed to spend more time thinking before producing a response, making them ideal for complex, multi-step problems.

GPT-5.2

The best model for coding and agentic tasks across industries

Price

Input:
\$1.750 / 1M tokens

Cached input:
\$0.175 / 1M tokens

Output:
\$14.000 / 1M tokens

OpenAI GPT-5.2



Max

Get the most out of Claude

From \$100

Per person billed monthly

Try Claude

Everything in Pro, plus:

- ✓ Choose 5x or 20x more usage than Pro*
- ✓ Higher output limits for all tasks
- ✓ Access Claude Code directly in your terminal
- ✓ Access to advanced Research
- ✓ Connect any context or tool through Integrations

Claude Max

Gemini 3 Pro Preview

gemini-3-pro-preview

Try it in Google AI Studio

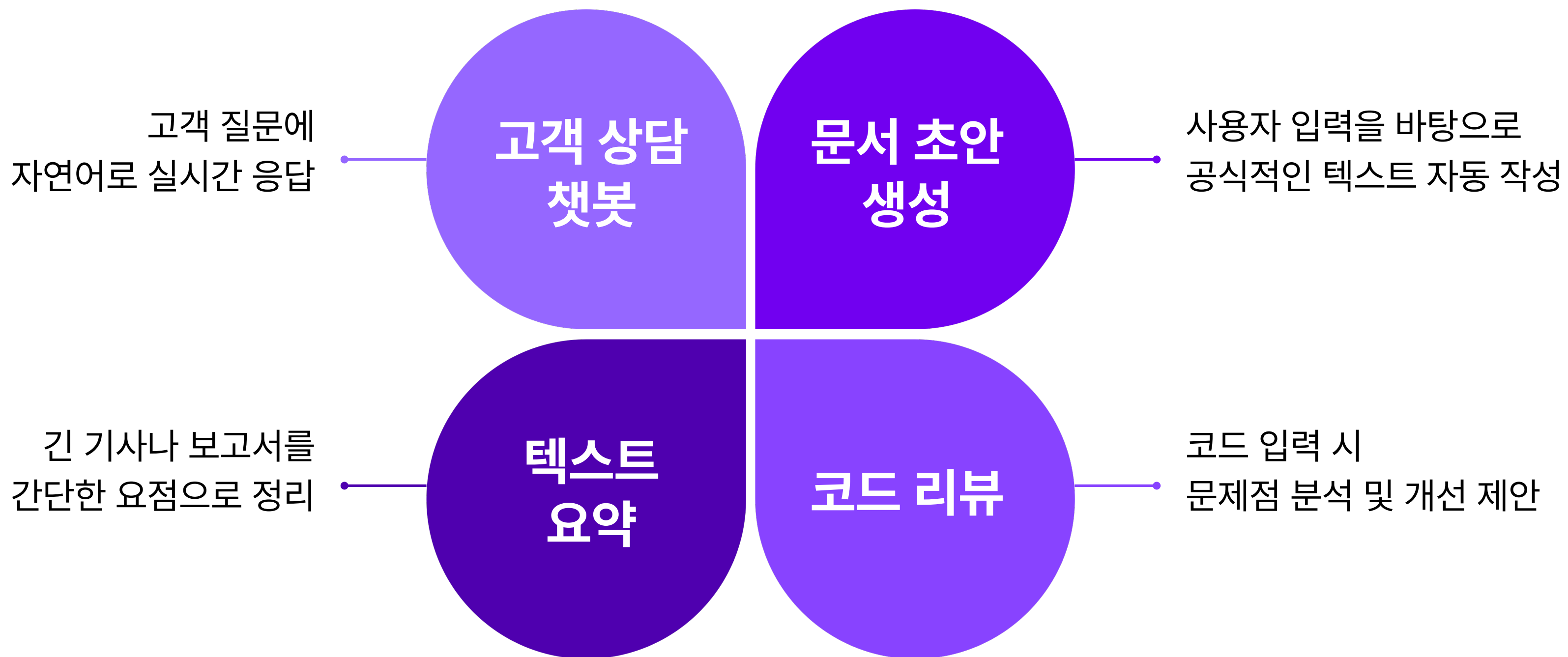
The best model in the world for multimodal understanding, and our most powerful agentic and vibe-coding model yet.

	Standard		Batch
	Free Tier	Paid Tier, per 1M tokens in USD	
Input price	Not available	\$2.00, prompts <= 200k tokens \$4.00, prompts > 200k tokens	
Output price (including thinking tokens)	Not available	\$12.00, prompts <= 200k tokens \$18.00, prompts > 200k	
Context caching price	Not available	\$0.20, prompts <= 200k tokens \$0.40, prompts > 200k \$4.50 / 1,000,000 tokens per hour (storage price)	
Grounding with Google Search*	Not available	5,000 prompts per month (free), then (Coming soon**) \$14 / 1,000 search queries	

Gemini 3 Pro



LLM API 활용 사례

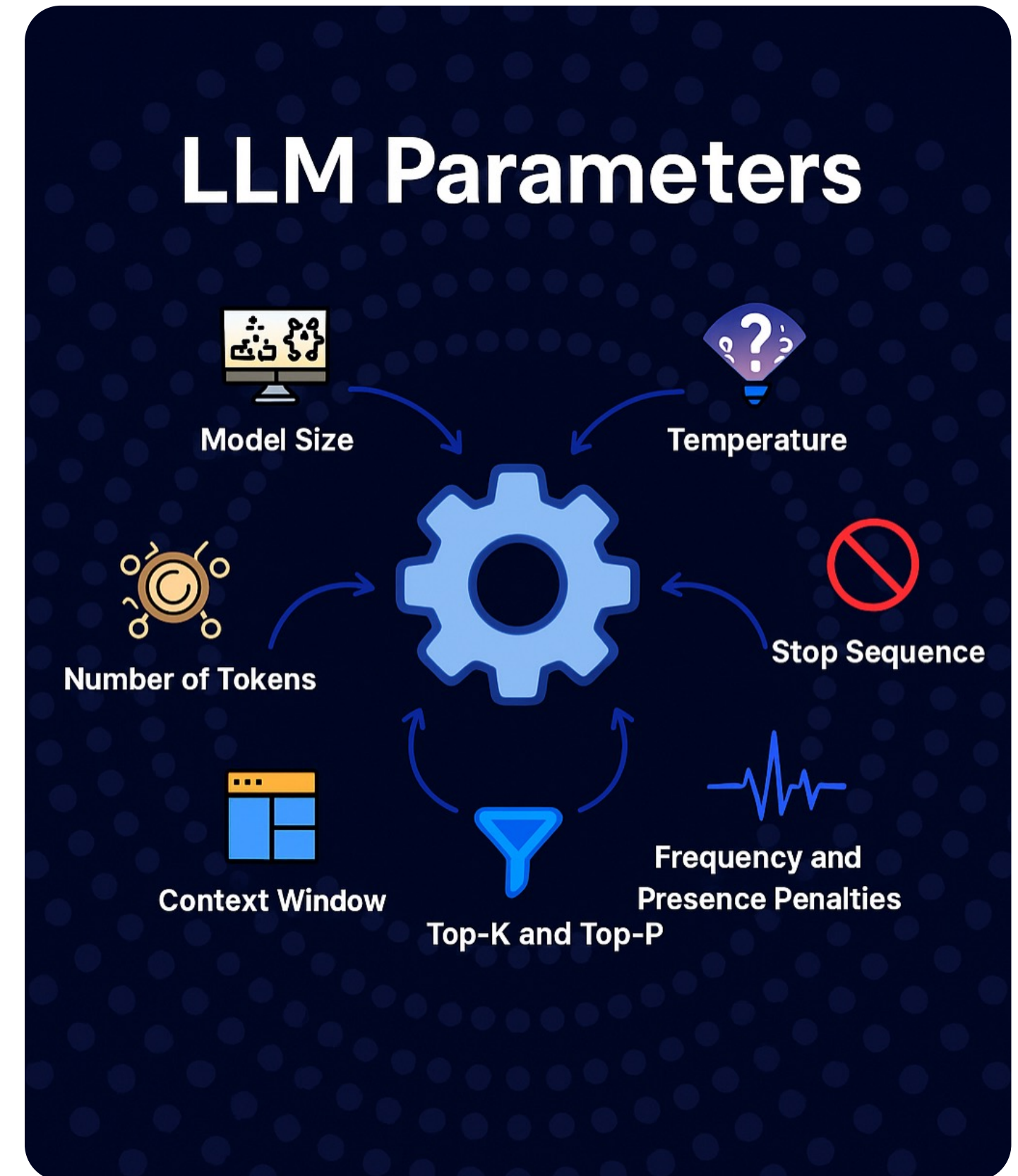


3. 하이퍼파라미터

파라미터 개념

프롬프트는 **무엇**을 물을지 결정 / 파라미터는 **어떻게** 대답할지를 결정

- LLM이 **어떻게** 응답을 생성할지 조정하는 설정값
 - 예: 얼마나 길게 말할까? 얼마나 창의적으로 답할까?
- 동일한 질문에도, 파라미터에 따라 전혀 다른 답변이 나올 수 있음
- 파라미터 설정은 **응답의 스타일, 길이, 창의성, 정밀도**를 결정함





Chat completion API 파라미터

- **Model** : 어떤 언어모델을 사용할지 선택
 - gpt-4o-mini 등
- **Messages** : 채팅 대화의 메시지 목록
 - 시스템 프롬프트와 유저 프롬프트 설정 가능
- **User** : api 요청한 사용자의 id 를 지정
- **Stream** : 응답을 스트리밍 형태로 출력

```
import openai

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "user",
         "content": "고양이에 대해 설명해줘"}
    ],
)
```

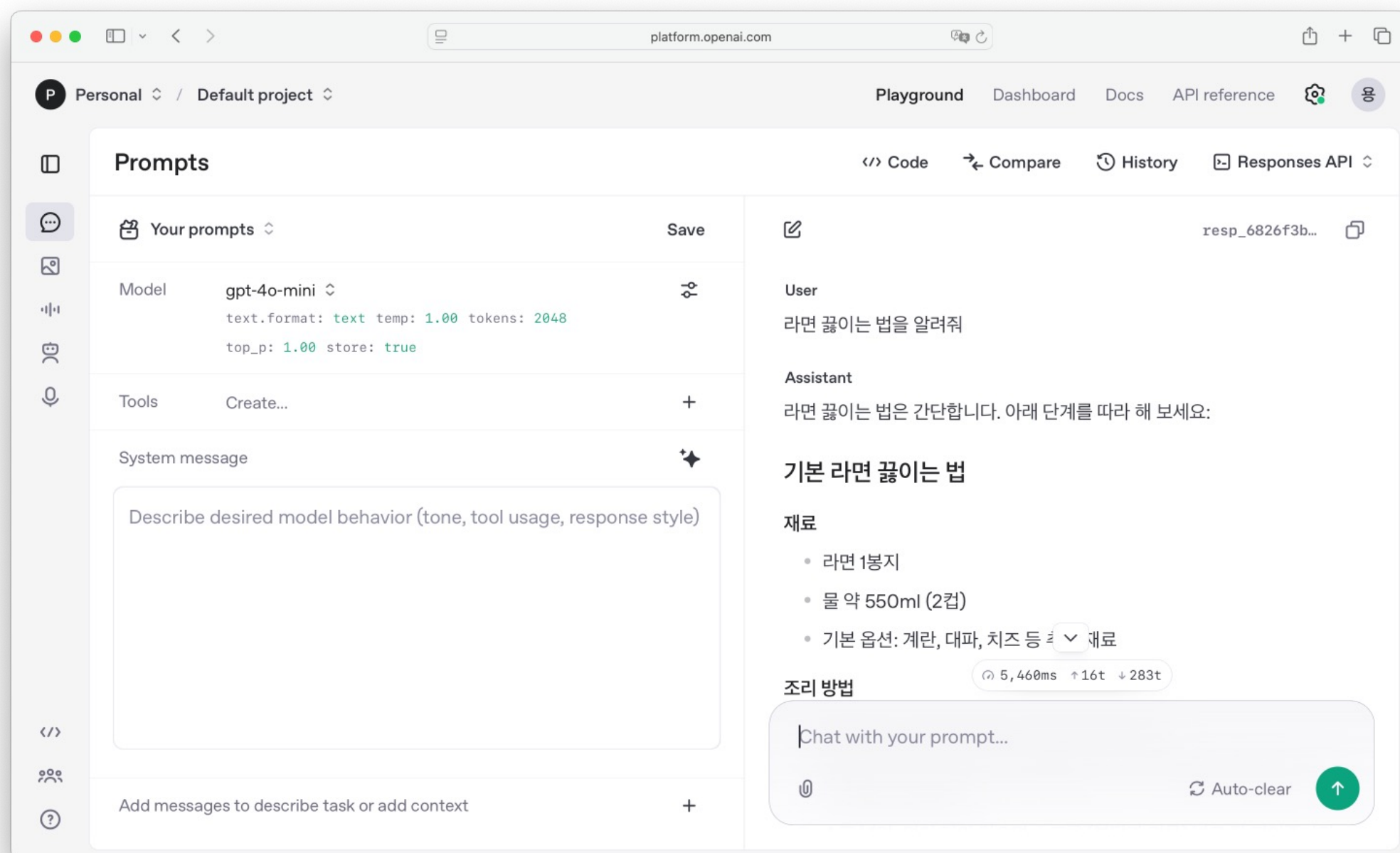


시스템 프롬프트 VS 유저 프롬프트

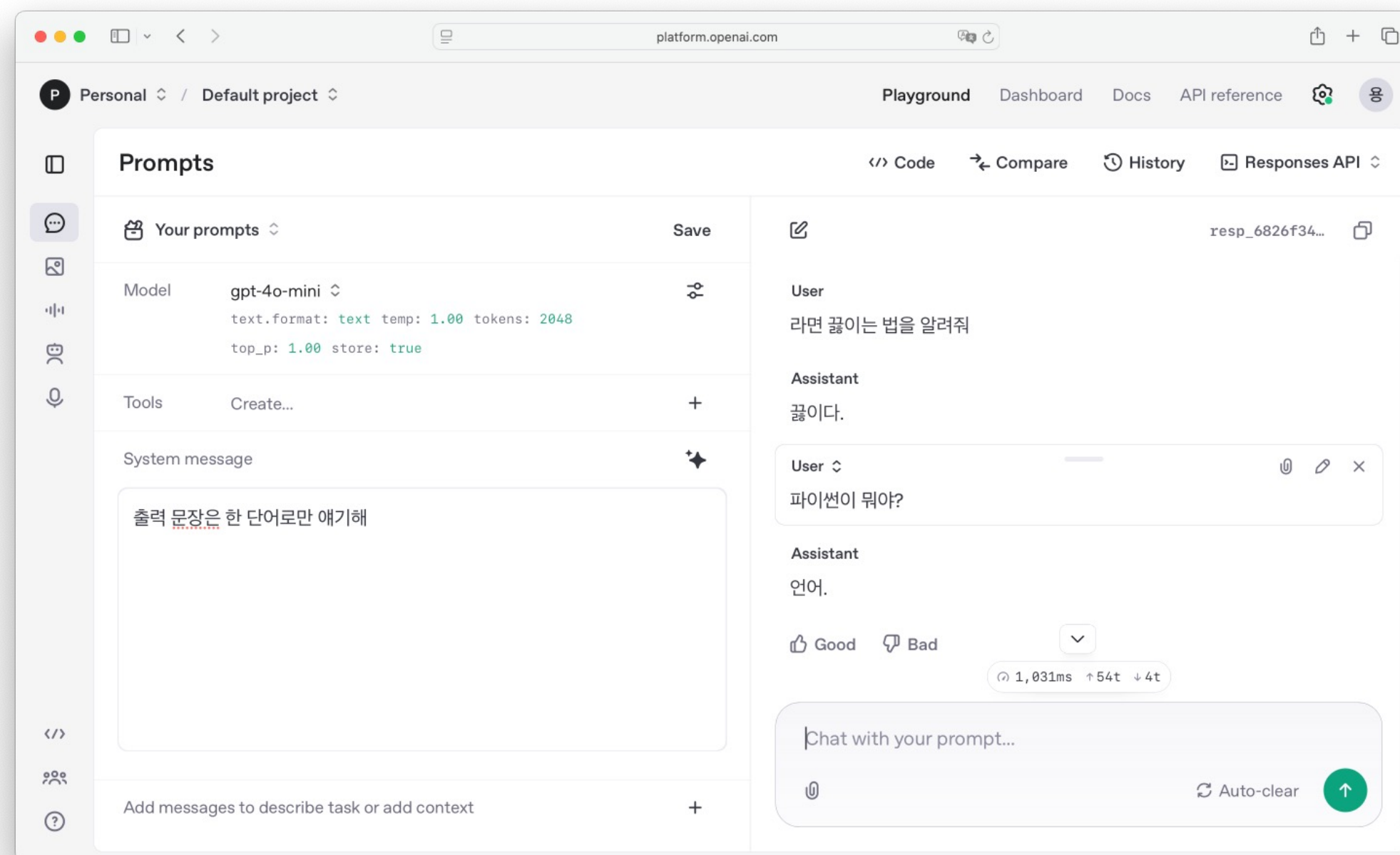
	시스템 프롬프트	유저 프롬프트
위치	대화의 시작 시점에 설정	사용자 입력 시 매 대화마다 전송됨
목적	모델의 성격, 말투, 역할, 응답 방향 설정	사용자의 실제 질문이나 요청 전달
지속성	대화 내내 유지됨	매 입력마다 새롭게 갱신됨
예시	"너는 친절한 여행 가이드야" → 모든 답변이 그 톤으로 유지	"프랑스에서 가장 추천할 만한 도시 알려줘"



하이퍼파라미터



시스템 프롬프트가 없는 상황



시스템 프롬프트를 추가한 상황
(답변을 한 단어로 제한)



시스템 프롬프트 / 유저 프롬프트

```
import openai
```

```
response = openai.ChatCompletion.create(
```

```
    model="gpt-4",
```

```
    messages=[
```

```
        {"role": "system",
```

```
         "content": "당신은 정중하고 예의 바른 비서입니다."},
```

```
        {"role": "user",
```

```
         "content": "오늘 서울 날씨 알려줘"}]
```

```
],
```

```
    temperature=0.7
```

```
)
```

```
print(response['choices'][0]['message']['content'])
```

시스템 프롬프트

모델의 **기본 행동 설정**을 담당

유저 프롬프트

실제 질문/명령을 전달

오늘 서울은 맑고 기온은 약 24도입니다.
우산은 필요 없겠습니다. 좋은 하루 보내세요!



시스템 프롬프트 설계 전략



역할 설정

모델이 누구인지
분명히 지정

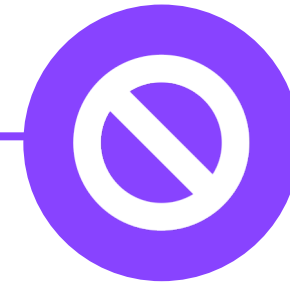
"너는 경험 많은
법률 상담사야"



응답 방식 안내

문체, 길이, 형식 등을
지정

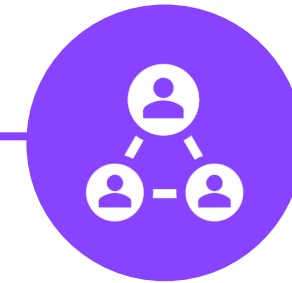
"항상 한 문단 이내로
요약해서 말해줘"



행동 제한 또는 의도 강조

금지 사항이나
강조해야 할 톤 지정

"절대 사적인 판단은
하지 마"



대상 독자 설정

초등학생, 전문가 등

"프로그래밍을 처음
배우는 중학생에게
설명하듯 말해줘"



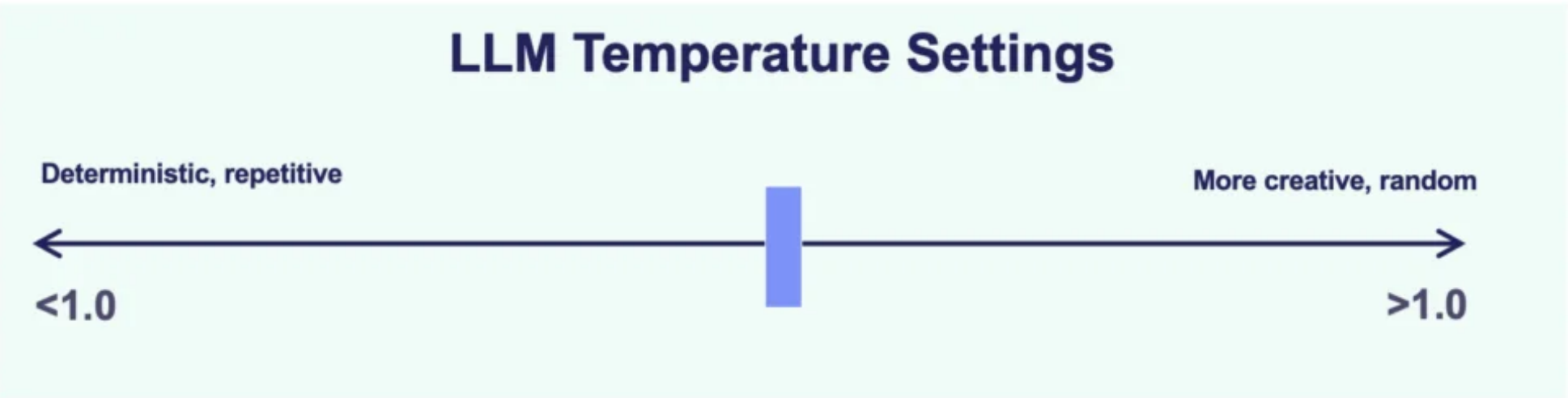
LLM 응답 제어 파라미터 (주요)

파라미터	설명	수치 예시	기능
temperature	랜덤성 조절	0.2 : 보수적 1.0 : 창의적	낮으면 다양성 감소 높으면 다양성 증가
top_k	확률 상위 K개 단어만 후보로 제한	top_k=1 : Greedy와 유사 top_k=50 : 상위 50개의 후보	후보군 수를 직접 제어
top_p	누적 확률 P 이하까지 포함된 단어만 후보로 사용	top_p=0.9 : 자연스러운 다양성 top_p=0.5 : 제한적	의미 있는 단어군 위주로 샘플링
max_tokens	응답에서 생성할 최대 토큰 수 제한	max_tokens=50 : 짧은 응답 max_tokens=300 : 상세 응답	응답 길이 제어
stop	특정 토큰을 만나면 응답 중단	stop=["\n", "User:"]	불필요한 후속 텍스트 방지

temperature

응답의 무작위성/창의성 제어

- 0~2 사이의 값 사용
- 낮은 temperature → 정확성 강화
- 높은 temperature → 창의성 강화



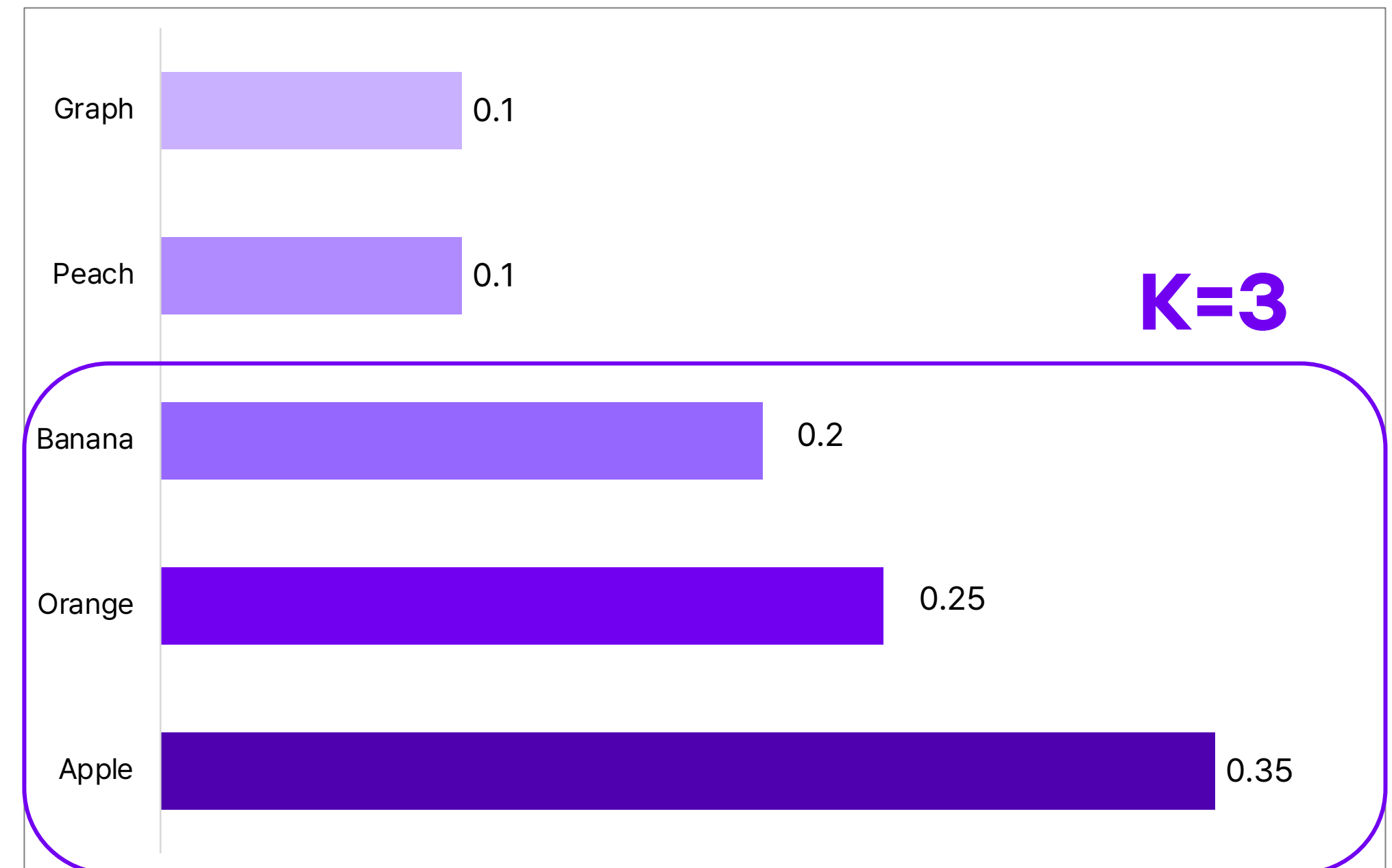
값	응답 특성
0.2	매우 보수적 정답만 추구
0.7	비교적 안정적이면서도 자연스러운 다양성 유지
1.0 이상	창의적 예측 불가능한 답변 가능
1.5 이상	무작위성 매우 높음 일관성 낮을 수 있음



top_k

매 토큰마다 확률이 높은 **상위 K개** 중에서만 선택

- K값이 작으면 예측 가능성 높음
- K값이 클수록 **다양성 증가**, 예상치 못한 답변 가능
- 예시
 - K=3 : 가장 가능성 높은 단어 3개 중 선택
 - K=1 : 가장 가능성 높은 단어 1개 선택
(= Greedy 선택 전략)

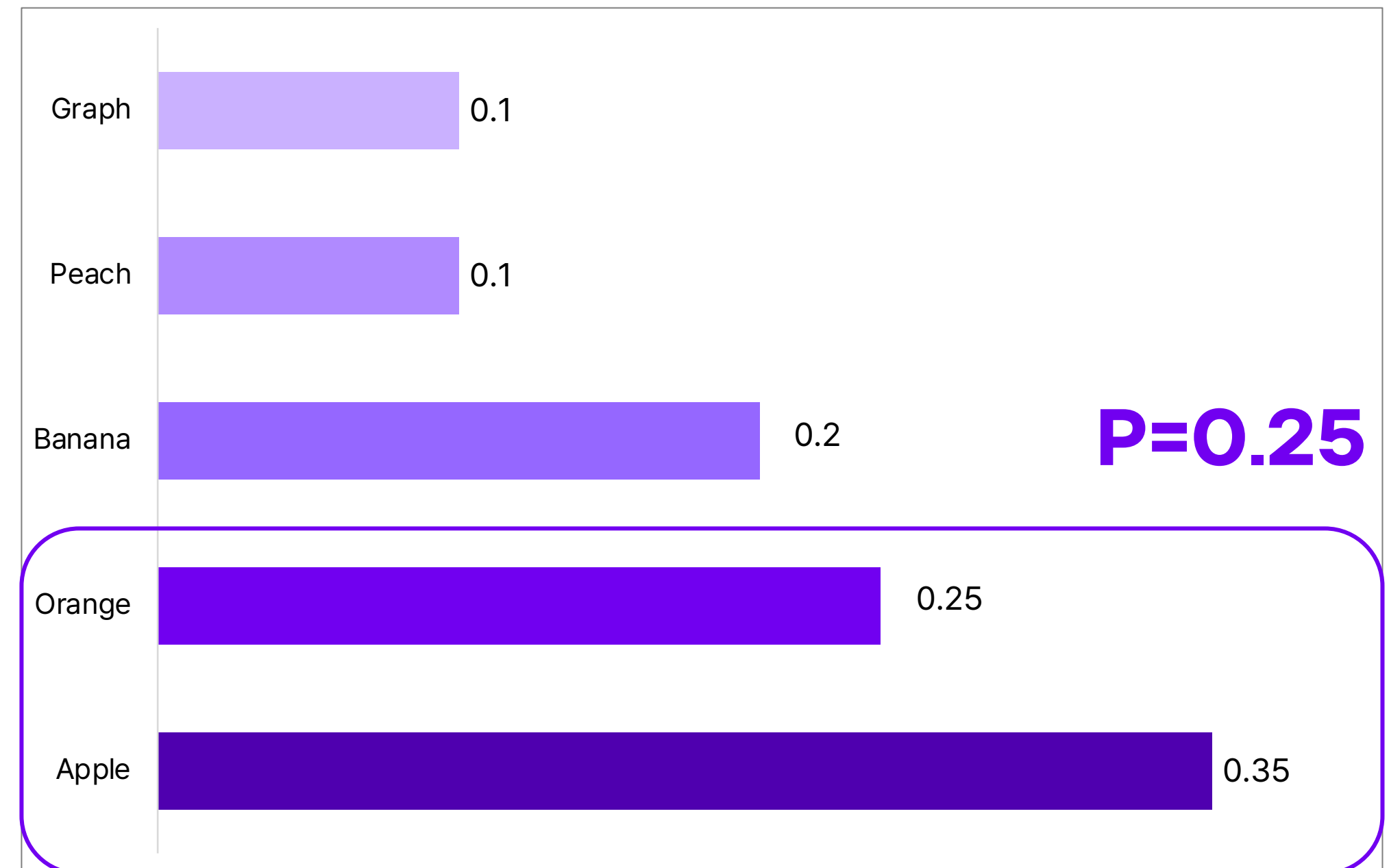




top_p

누적 확률이 P 이상의 상위 단어만 포함한 후보군 중에서 무작위 선택

- 확률이 낮지만 의미 있는 단어도 선택될 수 있음
- K처럼 고정 개수 대신 **누적 확률 기준으로 동적 조절**
- 예시
 - top_p=0.25 : 확률이 0.25 이상인 단어 중 무작위 선택

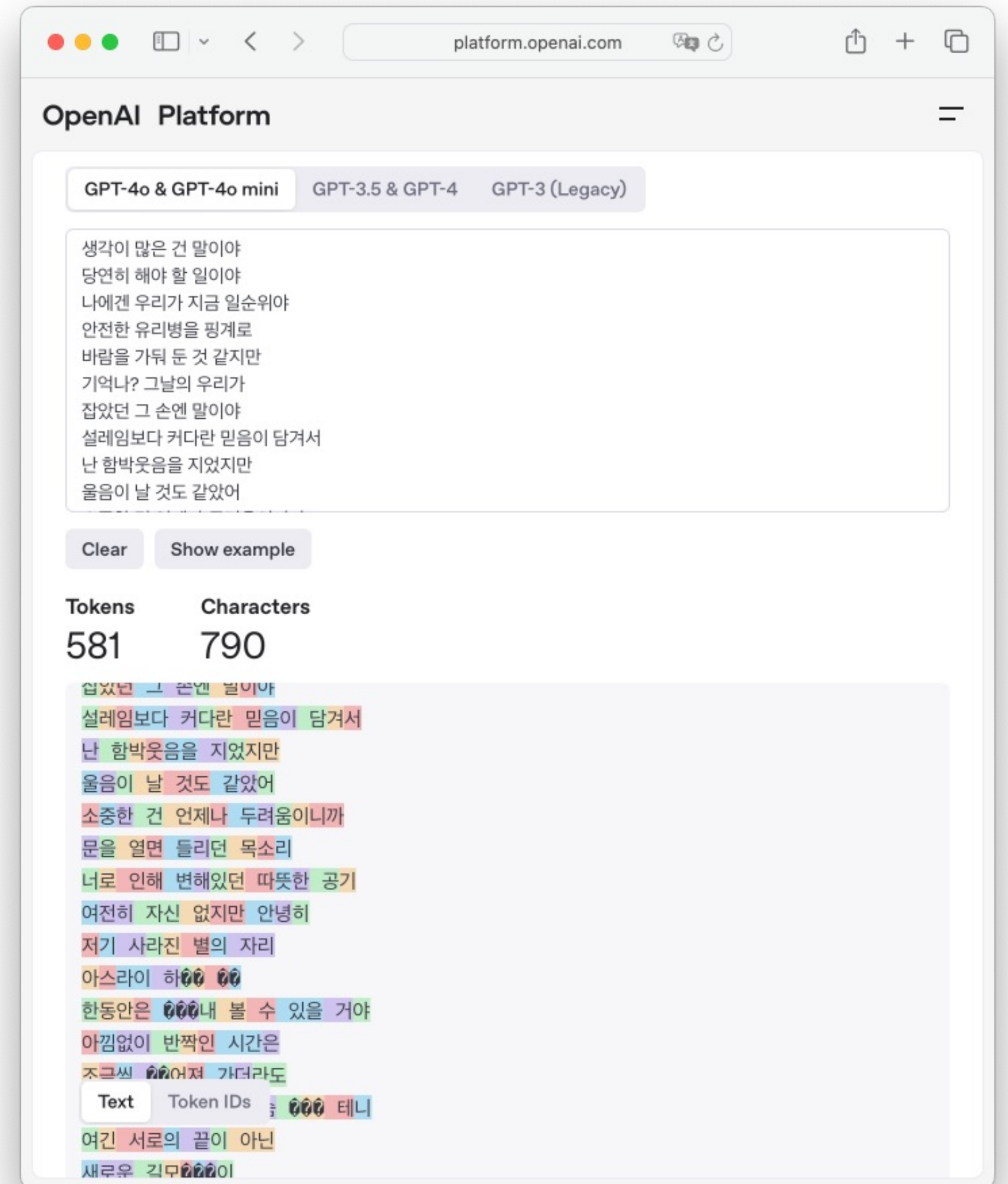


max_token

질문 + 응답문장의 최대 토큰을 조절

- 토큰 최대값은 **16,384** (gpt-4o mini 모델 기준)
- max_tokens 값을 조절하여 메모리를 관리할 수 있음
- max_tokens 값이 클수록 문장의 길이가 길어지지만, 품질이 저하될 가능성이 높아짐
- API 요청 토큰(질문 문장) $\leq$ max_tokens

토큰 (Token) : 모델이 처리하는 자연어의 기본 단위, 영어는 4글자=1토큰, 한글은 1글자=2~3토큰



Stop

특정 단어가 등장하면 응답 문장 생성을 멈춤

- Stop : ["초록"]
- 질문: 빛의 삼원색에 대해 알려줘
- 응답:

빛의 삼원색은 빛을 조합하여 다른 모든 색을 만들어내는 세 가지 기본색상입니다. 이 색상 모델은 빛의 혼합에 대해 설명하며, RGB (Red, Green, Blue) 색 모델로도 알려져 있습니다.

- 빨간색 (Red): 빨간색은 빛의 삼원색 중 하나로, 파장이 가장 긴 색상입니다. 빨간색 빛이 더 많을수록 색의 밝기와 빛의 강도가 증가합니다.

USER

빛의 삼원색에 대해 알려줘

ASSISTANT

빛의 삼원색은 빛을 조합하여 다른 모든 색을 만들어내는 세 가지 기본색상입니다. 이 색상 모델은 빛의 혼합에 대해 설명하며, RGB (Red, Green, Blue) 색 모델로도 알려져 있습니다.

- 빨간색 (Red): 빨간색은 빛의 삼원색 중 하나로, 파장이 가장 긴 색상입니다. 빨간색 빛이 더 많을수록 색의 밝기와 빛의 강도가 증가합니다.

+

Add message

Mode

Chat

Model

gpt-3.5-turbo

Temperature

1

Maximum length

256

Stop sequences

Enter sequence and press Tab

초록

Top P

1

Frequency penalty

0

Presence penalty

0

API and Playground requests will not be used to train our models. [Learn more](#)

Beam_width

값이 클수록 다양한 문장을 참조

- beam search 알고리즘을 기반으로 하는 하이퍼파라미터 (기본값 1)
 - beam search: 자연어 처리에서 사용되는 그래프 탐색 알고리즘
 - 가장 유망한 경로 n개를 고려
- 고려할 문장의 수를 기반으로, 숫자가 높을수록 다양한 문장 생성



Person A

nice, are they
doing that for a
particular reason?

yeah .

Beam Search
beam_width=10

yes and there are
typically on tv and be
happy to look at us,
that's a good question

Top-K Sampling
K=300, Temp=0.7

yeah. did you know
that 70s show was
remade in uk?

Nucleus Sampling
p=0.95

파라미터에 따른 결과 비교

- temperature=0.2 : 안정적인 답변
- top_p=0.95 : 의미 있는 후보 단어 위주로 선택
- max_tokens=50 : 비교적 짧은 응답 허용

고양이는 포유류에 속하는 동물로,
전 세계적으로 반려동물로 많이 길러집니다.
독립적이고 청결한 성격을 가지고 있으며,
야행성 습성이 있습니다.

```
import openai

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "user",
         "content": "고양이에 대해 설명해줘"}
    ],
    temperature=0.2,      # 창의력을 낮춤
    top_p=0.95,           # 확률 누적 기반 샘플링
    max_tokens=50,        # 응답 최대 길이
)

print(response['choices'][0]['message']['content'])
```



파라미터에 따른 결과 비교

- temperature=1.2 : 창의적이고 예측 불가능한 답변
- top_k=40 : 40개의 후보 단어 선정
- max_tokens=200 : 비교적 긴 응답 허용

고양이는 먼 우주에서 온 털 달린 요정 같아요.
그들은 인간의 집에 잠시 머무는
외계 생명체처럼 보이죠.
소파 위에 우아하게 앉아
인간을 심판하듯 바라보다가,
새벽 3시가 되면 갑자기 은하계를 정복하듯
집안을 질주해요.
...

```
import openai

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "user",
         "content": "고양이에 대해 설명해줘"}
    ],
    temperature=1.2,      # 창의력을 높임
    top_k=40,             # 후보 갯수 기반 샘플링
    max_tokens=200,       # 응답 최대 길이
)

print(response['choices'][0]['message']['content'])
```